Project Deliverable 2: Full Project Plan

CSE 4059: Applied Parallel Programming using GPUs

Group Members: Charlie Li, Jack Yang

1. Problem Description

The core computational problem is the calculation of non-bonded pairwise forces between a large system of N particles over discrete time steps. In a Molecular Dynamics (MD) simulation, every particle exerts a force on every other particle. In our specific model of a linear monopolymer, there are two classes of interactions formulated by the displacement vector $\vec{r}$ between two particles:

- Bonded Interactions modeled by the harmonic potential: $F_{bond}(\vec{r}) =- k(|\vec{r}| - r_0)\frac{\vec{r}}{|\vec{r}|}$, where $r_0 = 1$ is the equilibrium bond length
- Non-bonded interactions modeled by the Lennard-Jones potential, where ϵ is the energy well depth, and σ is the distance at which the potential energy is 0:
  - Attraction: $F_{attractive}(\vec{r}) =- \frac{24\epsilon_{attractive}}{|\vec{r}|}[(\frac{\sigma}{|\vec{r}|})^6 - 2(\frac{\sigma}{|\vec{r}|})^{12}]\frac{\vec{r}}{|\vec{r}|}$
  - Repulsion: $F_{repulsive}(\vec{r}) =- \frac{24\epsilon_{repulsive}}{|\vec{r}|}[(\frac{\sigma}{|\vec{r}|})^6 - 2(\frac{\sigma}{|\vec{r}|})^{12}]\frac{\vec{r}}{|\vec{r}|}$ when $|\vec{r}| < 2^{1/6}\sigma$ and 0 otherwise

The net force on each particle is the sum of all N-1 forces from other particles, which naively requires $O(N^2)$ computations.

MD simulates the trajectory of atom movements over time by repeatedly computing the forces, deriving acceleration by Newton's second law, and integrating over an extremely small timestep on the magnitude of femtoseconds. Since most other steps scale linearly with the number of atoms, the step of force computation is rate-limiting. To see notable motion of the system, a large number ($10^6 - 10^{12}$) of such timesteps need to be performed. Coupled with the large number of particles in a typical system, calculating the $O(N^2)$ forces for each step on a CPU quickly becomes computationally intractable.

Since the computation of forces on one atom does not affect that on another atom in a given timestep where their positions do not change, it is highly parallelizable. The goal of this project is to map this massively data-parallel workload onto the GPU to achieve a significant speedup.

2. Application and Application Domain

Polymer chains demonstrate complex physical phenomena, notably temperature-induced phase transitions between globular and extended configurations. These transitions are significant within biophysics, material science, and nanotechnology, where MD simulations provide a robust methodology for molecular-level investigation.

In space technology, linear monopolymers are widely used for thermal insulation and radiation shielding. However, the extreme temperatures in deep space causes polymers to fold and become brittle, which compromises their mechanical properties. MD is a great way of studying the influence of temperature on polymer folding and exploring its mechanism.

Past research has shown that polymer length significantly influences its phase transition behaviors in an MD simulation. Increasing the length from $N = 5$ to 80 causes a notable decrease in normalized radius of gyration at a given temperature, indicating a tendency of longer chains to stay folded under the same conditions. However, the sequential algorithm struggles with a system size of $N = 100$, while the length of a typical industrial polymer chain ranges from 2,000 to 10,000. Therefore, significant (20x - 100x) speedup of the MD algorithm is required to perform accurate simulations on real-life systems and draw physically sound conclusions.

3. Algorithms

We plan to implement the complete MD algorithm for a linear system, which includes force computations, position propagation, and velocity rescaling, among other parts. However, our emphasis is on force computation. In particular, we plan to implement two variants of the force computation algorithm.

The first one is the exact force computation. In this algorithm, each pair of forces is calculated, which guarantees the physical correctness of the simulation. However, this algorithm is computationally demanding, requiring $O(N^2)$ operations.

Another strategy of force computation is to define a cutoff radius beyond which the forces are negligible. Since the non-bonded Lennard-Jones interactions decay at a rate of $1/r^6$, at large distances, these interactions contribute little to the net force and can be effectively ignored. This strategy significantly reduces the computational complexity; since particles repulse in proximities, the number of particles within the cutoff radius is likely limited. Treating the number of particles within the cutoff distance as a constant, this algorithm requires only $O(N)$ operations. However, an energy correction term must be applied after the simulation to account for the truncated forces. In addition, depending on the specific implementation, which requires sorting of the atom positions to efficiently know which atoms are within the cutoff radius, the total time complexity could increase to $O(NlogN)$.

If time permits, we will also implement algorithms used in the analysis phase, which include determining the maximum pairwise distance and the radius of gyration.

4. Implementation Strategy

To implement the **exact force computation algorithm**, we observe that, by Newton's third law, the inverse of the force on one atom in an atom pair is the force on the other atom. We will follow the implementation strategy outlined in [1], where the authors formulated the computation as a symmetrical force matrix. The upper triangular part of the matrix is separated into tiles of size p*p, as shown in figure 1. Each block of p threads maps to one tile, loads the positions of the atoms corresponding to the columns of the matrix (atoms receiving the force, or the receivers) into registers and the positions of the row atoms (atoms exerting the force, or the sources) into shared memory. Each block keeps a running sum of the net

force on the receivers along the vertical direction in the registers, as well as that on the sources along the horizontal direction. After the partial sums are complete, each thread performs two atomic additions to accumulate the forces of both the column and the row atoms to the global forces array.

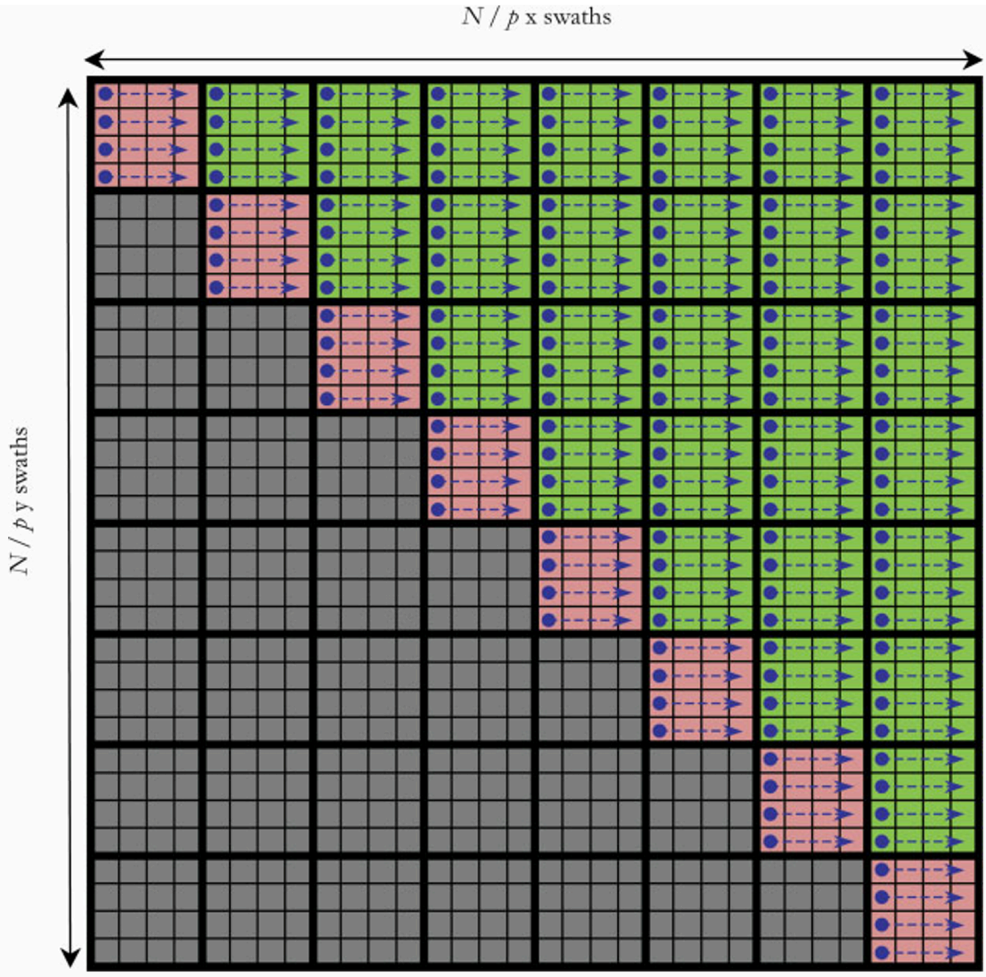

Figure 1. The set of *p*x*p* tiles required for force calculation-only the pink and green tiles need to be calculated. Force data for the grey tiles can be generated by negating the sign of the forces calculated for the corresponding green tile on the other side of the pink diagonal.

To minimize global atomic additions, we will use the technique of privatization, where each block first accumulates private sums in registers or shared memories. To further reduce contention in shared memory, we juxtapose the positions processed by neighboring threads. In iteration i, each thread will write to position (i+threadIdx)%p in shared memory, which maximally reduces conflict. When p = 32, which is the warp size, there will be maximum memory traffic without contention.

To implement the **computation with a cutoff radius**, we will divide the unit cell into a grid of cubes, each with a side length greater than or equal to the cutoff. We observe that for any given atom inside one cube, all the atoms that could potentially interact with it must reside within its cube or its neighboring 26 cubes. We will first launch a kernel to assign a cube index to each atom, then sort the positions of the atoms based on the index assignment. After that, we will launch another kernel to obtain the boundary indices of each segment of the positions array corresponding to different cubes. Finally, a kernel is launched to calculate the force on each atom.

The final kernel will use shared memory to reduce repeated global accesses. We will map thread to each atom, which coordinately loads a segment of the positions into shared memory. Since the positions have been sorted, the memory accesses will be coalesced. Each thread then accumulates the forces from that segment of atoms into their registers. This process repeats until all atoms in neighboring cubes have been considered.

In both strategies, contiguous access to each component of the positions array is desired. Therefore, we will store our positions array in a layered format with a shape of $3 \times N$, keeping the coordinates of each component in a contiguous array. This enables memory coalescing.

5. Anticipated Obstacles

In the design process of our algorithms, we encountered several obstacles. The first obstacle is how we should modify the algorithm so that it better fits the GPU architecture. Our first thought was whether we could simply parallelize across each timestep of the simulation. The answer is clearly **NO**, because the positions and velocities of atoms at a previous timestep will directly affect where they move in the next timestep. This means the computation is highly dependent on the time dimension. Because of this, we started to look at how we could parallelize the computation within each timestep instead.

Within each timestep, we need to compute the net force applied to each atom, which requires all pairwise forces to be considered. At the beginning, we proposed one possible strategy where the kernel only computes the pairwise force matrix, which is an $N \times N \times 3$ tensor, and then on the host side we use a vectorized reduction such as sum to compute the final net force on each atom. This strategy would separate the computation into cleaner parts, so the GPU could fully focus on pairwise force calculation and the code structure would also be simpler. However, this strategy has a significant memory problem. The performance would be bottlenecked by the creation of a very large global tensor to store all intermediate forces, and there is no efficient way to avoid the memory bandwidth cost in this approach. For realistic system sizes, this intermediate storage would be too expensive and could remove most of the benefit of GPU acceleration.

This leads to our current strategies described in Sections 3 and 4. In designing the exact force computation strategy, one major obstacle is how to reduce the overhead of atomic operations. This problem comes from the fact that multiple threads may try to write to the same net force entry at the same time, since we need to reduce a $N \times N \times 3$ force representation to an $3 \times N$ net force array. Threads that process interactions involving the same row or column atom may all contribute to that atom's final force simultaneously. If every update directly writes to global memory, the performance may become close to linear or even worse because of the locking and serialization overhead. To deal with this problem, we decided to use shared memory, as described in Section 4, to first compute partial sums inside each tile and then use atomic additions only for those partial sums. In this way, the number of atomic operations can be reduced significantly.

Another obstacle is the memory access pattern. Even if the arithmetic part of the computation is parallel, the overall performance can still be limited by inefficient memory access. If the positions of atoms are not stored in a layout that matches the GPU access pattern, then global memory accesses may not be coalesced, which will reduce performance. Because of this, we plan to store the positions in a layered format so that each coordinate component is stored contiguously. This should improve coalesced memory

access, but we still need to further investigate the tradeoff between shared memory usage, register usage, and occupancy.

For the cutoff-based algorithm, there are also additional obstacles that need further investigation. Although the cutoff idea can reduce the number of force computations significantly, it introduces extra preprocessing steps, including assigning atoms to spatial cubes, sorting them by cube index, and finding the boundary indices of each cube segment. These extra steps also cost time. If the overhead of sorting and preprocessing is too large, then the overall speedup may not be as significant as expected, especially for medium-sized systems. So one obstacle is how to design the cutoff strategy in a way that the reduction in force computation is worth the added preprocessing cost.

There are also obstacles related to the actual force formulas used in the simulation. As described in Section 1, we have two different forces for different cases. If two atoms are bonded neighbors, we use the harmonic force. Otherwise, we use the Lennard-Jones force. How to compute these two forces efficiently will be another potential bottleneck of our simulation. If both calculations are performed inside the same kernel using conditional branches, this may create control divergence between threads in the same warp. On the other hand, if we precompute some force-related values or separate the bonded and non-bonded interactions into different kernels, that may reduce divergence but increase implementation complexity or memory cost. Whether it is better to compute these forces directly in the kernel or reorganize them into separate steps is still a problem we need to investigate.

Finally, another obstacle is how to balance computational efficiency and physical accuracy. The exact force computation preserves all pairwise interactions, but it is computationally expensive. The cutoff-based method is faster, but it introduces approximation error and may require additional correction terms after the simulation. Because of this, we not only need to evaluate the speedup of the GPU implementation, but also check whether the results are still physically reasonable. Therefore, one important challenge of this project is to find a strategy that gives significant performance improvement while still maintaining acceptable simulation accuracy.

6. Project Timeline

Since we only have four weeks left, our timeline needs to focus on the main parts of the project and leave enough time for testing and final analysis. By **April 9th**, we will implement the two force computation strategies described in this report. Charlie and Jack will each be responsible for one strategy and make sure the algorithm is runnable and has tolerable discrepancies compared to the sequential version. This will also be turned in as the checkpoint writeup.

Once the implementation is correct, we will begin measuring runtime and identifying the main performance bottlenecks, especially those related to atomic operations and memory access. We expect this stage to take one or more weeks, depending on the debugging and testing results.

Starting **April 20th**, we will move to final optimization, profiling, and drafting the final write-up while also preparing for the presentation video. We will tune kernel parameters, improve memory access patterns where possible, and collect the final runtime and accuracy results for both strategies. If time permits, we may also include additional analysis such as radius of gyration and end-to-end distance.

Finally, we will organize the results, prepare figures, and complete the final write-up and video for submission by **April 30th**.

References

[1] M. S. Friedrichs *et al.*, "Accelerating molecular dynamic simulation on graphics processing units," *Journal of Computational Chemistry*, vol. 30, no. 6, pp. 864–872, Apr. 2009, doi: https://doi.org/10.1002/jcc.21209.